Cordell Programming Language: Typed OS Interfaces in a Compact SSA-Based Compiler

Nikolai Fot
Alexander Vinarskiy
Compiler Technology Department,
Ivannikov Institute for System Programming of the Russian Academy of Sciences (ISP RAS)

June 13, 2026

Abstract

This report investigates whether a compact systems language can expose the binary layout and machine-interface controls required by freestanding software while retaining a conventional optimizing pipeline and SSA-based static diagnostics. Cordell Programming Language (CPL) provides primitive values, pointers, arrays, aggregates and unions, entry-point and section control, system calls, and inline assembly without implementing the full semantics of C. Three questions structure the study: whether typed CPL can express representative kernel boundaries with assembly confined to irreducibly machine-specific operations; whether its generated code is comparable to mature compilers on selected small kernels; and whether SSA plus SMT provides a useful basis for path-sensitive diagnostics. Compiler version 3.6.4 is examined through its HIR/LIR pipeline, Z3 integration, PTRN peephole language, x86-64 and i386 backends, a Multiboot bootstrap case study, and three microbenchmarks. The case study demonstrates typed expression of boot-header layout, section placement, stack allocation, and kernel transfer. CPL matches or approaches Clang and GCC on the two arithmetic-loop kernels reported here, but trails all compared compilers on pointer-heavy string traversal. The diagnostic architecture is demonstrated functionally, but precision and scalability are not quantified. The evidence therefore supports feasibility, not general performance, correctness, or diagnostic-effectiveness claims.

1 Introduction

Systems programming languages expose machine-level details while attempting to keep programs tractable. Mature languages such as C [13, 7], C++ [14, 15], Rust [8, 6], and Zig [18] provide large ecosystems, but their full syntactic and semantic complexity makes them expensive targets for compiler experiments. Conversely, many teaching compilers are intentionally small but do not expose enough low-level mechanisms to study system-oriented code generation, platform-specific interfaces, and optimization under realistic constraints.

CPL is not a C subset. A faithful C-like subset would either require a substantially more complex parser and semantic model, or would create misleading expectations that the rest of C should also be supported. The current design instead favors a compact parser-friendly grammar, explicit syntax for functions and pointer-like operations, and compiler-supported low-level constructs.

The central research problem is the tension between implementation compactness and systems-level adequacy. Reducing a language and compiler makes experimentation and inspection easier, but can remove the layout controls, ABI mechanisms, analyses, and backend structure needed for realistic low-level programs. This report studies how far a compact implementation can preserve those capabilities without claiming the safety, verification, or maturity of established systems toolchains.

The contributions of this report are:

- a compact systems-language design combining typed data layout with explicit entry-point, section, register, syscall, and inline-assembly controls;
- a multi-stage implementation with HIR, SSA, LIR, instruction selection, register allocation, and generated peephole rewrites;
- a qualitative Multiboot case study showing which bootstrap responsibilities can be represented in typed CPL and which remain assembly-specific;
- preliminary performance evidence that distinguishes arithmetic-loop behavior from pointer-heavy behavior;
- an SSA/SMT diagnostic architecture whose demonstrated capabilities and unmeasured properties are reported separately.

Unless stated otherwise, implementation claims refer to compiler version `3.6.4:1106.26` and the source snapshot dated June 13, 2026. This explicit snapshot boundary is necessary because the language and compiler are under active development.

2 Research Questions and Scope

RQ1: Systems expressiveness. To what extent can a compact typed language represent freestanding kernel interfaces that are conventionally written in assembly? The Multiboot case study evaluates header layout, section alignment, linker-visible entry points, stack construction, boot-register capture, and transfer to a kernel routine. Evidence is qualitative and artifact-based; no claim is made that CPL eliminates assembly.

RQ2: Generated-code performance. How does optimized CPL code compare with Clang, GCC, and Rust on small kernels that stress loop overhead, scalar recurrence, and pointer traversal? The available evidence consists of three five-run microbenchmarks. It can identify workload-specific strengths and weaknesses but cannot establish whole-program competitiveness or statistical significance.

RQ3: SSA/SMT diagnostics. Does SSA-form HIR provide a workable representation for path-sensitive Z3 queries beyond AST-local diagnostics? The report demonstrates null and reachability query construction. Analysis time, false-positive rate, and false-negative rate have not been measured, so diagnostic effectiveness remains an open empirical question.

RQ4: Optimization attribution. Which HIR and LIR passes account for changes in execution time and code size? The current measurements compare optimization profiles but do not provide pass-by-pass ablation. RQ4 is therefore stated to define the required experiment and is not answered by the present dataset.

2.1 Design Scope

The language stays close to assembly and C by exposing pointers, explicit dereferencing, C-like containers, direct system calls, inline assembly, manual entry points, and annotations for placement and control. It deliberately avoids reproducing the full C grammar and semantics. The compiler includes enough phase separation to study SSA construction [3], diagnostics, scalar optimization, low-level selection, register allocation, and peephole rewriting.

CPL minimizes dependencies on a host standard library. In the evaluated snapshot, x86-64 Mach-O is the default and most extensively exercised backend. The x86-64 Linux and i386 Linux paths are implemented and covered by targeted tests, although their validation remains narrower.

2.2 Non-Goals

CPL does not claim any of the following properties:

- formal verification of the compiler;
- memory safety or ownership checking;
- production readiness of CPL as a general-purpose language;
- source-level compatibility with C;
- a mechanized semantics or compiler-correctness proof;
- a general-purpose standard library or package ecosystem.

CPL supports lightweight containers as C-like aggregates, but deliberately avoids classes, constructors, destructors, inheritance, traits, and ownership-based object models. This keeps the language focused on primitive values, pointers, arrays, containers, functions, and explicit low-level constructs.

3 Language Overview

3.1 Program Entry

A CPL program can use a `start` block as its static entry point. The entry block does not return a value with `return`; it must terminate through `exit`. An ordinary function can also become an entry point through the `@[entry]` annotation.

```
start() {
    exit 0;
}
```

Listing 1: Minimal CPL entry point.

The entry point may accept arguments, such as `argc` and `argv`, and can be annotated for platform-specific entry behavior. The `naked` annotation disables default entry and exit routines, which is useful for system-level code that must manage its own prologue and epilogue. On i386 this also affects how stack arguments are addressed, as discussed in Section 4.4.

3.2 Types

CPL uses permissive static typing. Variables do not dynamically change type, widening conversions may be inserted implicitly, and narrowing conversions require an explicit `as` cast. The primitive integer and floating-point types include `i8`, `u8`, `i16`, `u16`, `i32`, `u32`, `i64`, `u64`, `f32`, `f64`, and the void-like function return type `i0`. Boolean-like logic follows the C convention: zero is false and non-zero is true.

3.3 Pointers, Strings, and Arrays

The language exposes pointer types through the `ptr` modifier. The `ref` keyword obtains a pointer to an object or string literal, and `dref` reads or writes through a pointer.

```
i32 x = 123;
ptr i32 p = ref x;
i32 y = dref p;
dref p = y + 1;
```

Listing 2: Pointer operations in CPL.

Strings and arrays are distinct built-in containers. A `arr[0, i8]` allocates a terminated byte sequence, while `arr` declares a fixed-size array. String literals placed independently in code reside in a read-only section-like area and must be referenced explicitly.

```
arr msg[0, i8] = "Hello world!";
arr xs[4, i32] = { 1, 2, 3, 4 };
ptr i8 p = ref "Hello, World!\n";
```

Listing 3: Strings and arrays.

3.4 Containers

CPL containers are lightweight C-like aggregate types. They group fields of different types in one named layout and are intended for explicit systems code rather than for object-oriented programming.

```
container pair {
    i32 first;
    i32 second;
}
```

Listing 4: A simple CPL container.

By default, container fields use the target architecture's default maximum alignment policy. A container can override this with the `align` annotation. For example, `@[align(1)]` requests a packed layout, which is useful when a byte-exact representation is required.

```
@[align(1)]
container packed_value {
    i8 a;
    i16 b;
    i32 c;
} :/ packed, occupies 7 bytes on the stack /:
```

Listing 5: Packed container layout.

In addition to fields, a container may declare functions. These functions are ordinary CPL functions attached syntactically to the container definition. They do not make the container a class, and they do not introduce implicit constructors or destructors.

```
container counter {
    function default_value() -> i32 {
        return 100;
    }
}

counter c;
c.default_value();
```

Listing 6: Container function.

The `self` annotation marks a container function whose first explicit argument is the receiver pointer. Calls to such functions may use method-like syntax, and the parser can pass the container pointer automatically. This is syntax sugar only: a `self` function must still declare its receiver argument explicitly.

```
container counter {
    i32 val;

    @[self]
    function init(ptr counter self) -> i0 {
        self.val = 0;
    }
}

counter c;
c.init();
c.val; :/ 0 /:
```

Listing 7: A self container function.

Containers should therefore be treated as modified C structures with optional attached functions. Initialization, cleanup, and ownership-like behavior must be written explicitly by the programmer.

Containers may also represent unions. With `@[union]`, all fields share offset zero and the aggregate size is determined by its largest field together with the selected alignment policy. The annotation composes with `@[align]` and `@[like_c]`.

```
@[like_c]
@[union]
container device_value {
    i8  size8;
    i16 size16;
    i32 size32;
    i64 size64;
}
```

Listing 8: Union container with C-compatible layout.

Container values can be array elements, and array element types may themselves be arrays. These forms permit arrays of records and statically sized matrices without introducing a separate aggregate system.

```
container string_view {
    ptr i8 data;
    i32 length;
}

arr views[10, string_view];
arr matrix[10, arr[10, i32]];
```

Listing 9: Arrays of containers and nested arrays.

A container method may be declared in the container and defined separately with the `::` qualifier. This form avoids emitting the same implementation in every translation unit while retaining container-qualified lookup.

```
container math {
    glob function add(i32 a, i32 b) -> i32;
}

function math::add(i32 a, i32 b) -> i32 {
    return a + b;
}
```

Listing 10: Out-of-container method definition.

3.5 Control Flow

CPL provides `if`, `while`, `loop`, and `switch`. A semicolon separates the condition from the body in conditional constructs. The `loop` construct represents an infinite loop unless terminated with `break` or annotated with `@[counter(n)]`. The `switch` construct supports fallthrough by default, while `@[no_fall]` inserts break-like behavior into cases.

```
if x > 0; {
    x -= 1;
}
else {
    x = 0;
}

@[counter(10)] loop {
    x += 1;
}
```

Listing 11: Control flow.

3.6 Functions

Functions are declared with the `function` keyword. CPL supports prototypes, default arguments, local functions, function overloading with restrictions, generic functions, function pointers, lambdas without closure capture, and variadic arguments. Global functions use `glob` when their symbol name must be preserved for external linkage.

```
function add(i32 a, i32 b = 1) -> i32 {
    return a + b;
}

glob function exported(i32 x) -> i32;
```

Listing 12: Function forms.

Function pointers have no signature-level static information. This keeps them flexible but limits static checking, overload resolution, and default-argument insertion after a function is stored as a raw pointer.

3.7 System-Level Facilities

CPL provides two built-in low-level mechanisms: `syscall` and `asm`. A `syscall` expression is target-dependent and accepts a platform-specific argument list. Inline assembly supports argument substitution through numbered placeholders.

```
i32 a = 0;
i32 ret;
asm(a, ret) {
    "push rax",
    "mov rax, %0",
    "syscall",
    "mov %1, rax",
    "pop rax"
}
```

Listing 13: Inline assembly with placeholders.

Inline assembly is intentionally powerful and fragile. It is copied close to directly into the generated assembly after placeholder substitution, so the programmer must preserve registers and match the selected target syntax. The compiler does not optimize inside inline assembly blocks.

3.8 Annotations

Annotations extend the small grammar with low-level behavior. Table 1 summarizes the system-oriented annotations currently available in the implementation. The maturity column separates stable annotations from annotations whose behavior is still target-dependent or known to be experimental.

These annotations expose system-level control without introducing an object-oriented runtime model. The `like_c` annotation is relevant for containers that cross a C ABI boundary because it requests C-compatible padding assumptions. The `section` annotation accepts an optional alignment argument, for example `@[section(".bss", 16)]`; this is distinct from aligning an individual declaration because the emitted section directive itself carries the alignment constraint. The `only_body` annotation supports target directives or instruction sequences that must appear without a generated function label, while preserving the function body as an optimizable compiler object before emission.

3.9 Core Static and Dynamic Model

The complete language is defined by the implementation, but the safety-relevant core used in this report can be stated independently. Let primitive types be $b \in \{\texttt{i8}, \ldots, \texttt{u64}, \texttt{f32}, \texttt{f64}, \texttt{i0}\}$, and let

$$\tau ::= b \mid \texttt{ptr}\ \tau \mid \texttt{arr}[n, \tau] \mid C,$$

where C is a container type. A typing environment Γ maps identifiers to types, a container environment Δ maps field pairs $C.f$ to field types, and $\Gamma \vdash e : \tau$ denotes expression typing.

$$\frac{\Gamma \vdash x : \tau}{\Gamma \vdash \texttt{ref}\ x : \texttt{ptr}\ \tau} \qquad \frac{\Gamma \vdash e : \texttt{ptr}\ \tau}{\Gamma \vdash \texttt{dref}\ e : \tau},$$

$$\frac{\Gamma \vdash e : C \qquad \Delta(C, f) = \tau}{\Gamma \vdash e.f : \tau}.$$

Widening numeric conversions may be inserted by the compiler; narrowing conversions require an explicit `as` expression. Pointer dereference is type-correct when its operand has pointer type, but typing does not establish allocation validity or non-nullness. This distinction motivates the separate SSA/SMT diagnostic layer.

A small-step memory model uses a store σ from addresses to typed values and locations ℓ:

$$\langle \sigma, \texttt{ref}\ x \rangle \to \langle \sigma, \ell_x \rangle, \qquad \frac{\sigma(\ell) = v}{\langle \sigma, \texttt{dref}\ \ell \rangle \to \langle \sigma, v \rangle}.$$

For assignment through a pointer,

$$\langle \sigma, \texttt{dref}\ \ell = v \rangle \to \langle \sigma[\ell \mapsto v], v \rangle.$$

Container field access is reduced by adding the target-dependent field offset to the base location; union fields have offset zero. Function calls evaluate arguments, establish parameter bindings, execute the body, and yield the explicit return value. Inline assembly, syscalls, and target annotations are modeled as externally defined transitions because their behavior depends on the selected ABI and machine. These rules define the report's core reasoning model, not a mechanized semantics or a proof that every compiler pass preserves it.

Table 1: Selected CPL annotations.

Annotation	Purpose	Example	Maturity
`naked`	Disable default entry and exit routines.	`@[naked] start()`	stable
`align`	Align a local or global object.	`@[align(16)] glob i32 a;`	stable
`section`	Place code or data in a target section.	`@[section(".data")]`	stable
`nosection`	Emit a declaration without an explicit section directive.	`@[nosection] glob function f()`	stable
`entry`	Mark a function as an entry point.	`@[entry("_start")]`	stable
`no_fall`	Insert break-like behavior in switch cases.	`@[no_fall] switch x;`	stable
`straight`	Use linear switch selection.	`@[straight] switch x;`	stable
`counter`	Generate a counted loop.	`@[counter(100)] loop`	stable
`hot/cold`	Influence branch layout.	`@[cold] if cond;`	stable
`register`	Bind a primitive variable to a register index.	`@[register(RAX)] i32 x;`	stable
`self`	Mark a container method whose explicit receiver pointer can be passed through method-like syntax.	`@[self] function init(ptr T self)`	stable
`poparg`	Influence argument popping at a low-level call boundary.	`@[poparg]`	stable
`like_c`	Request C-ABI-like padding and layout behavior for a container.	`@[like_c] container T {...}`	stable
`union`	Give all fields of a container offset zero.	`@[union] container U {...}`	implemented
`abi`	Mark an external function declaration as ABI-compatible.	`@[abi] extern function f(...)`	implemented
`weak`	Emit a weak linker symbol for a function.	`@[weak] function f()`	implemented
`only_body`	Emit only the lowered body of a function, without its symbol label or ordinary wrapper.	`@[only_body] function h()`	experimental

4 Compiler Architecture

4.1 Pipeline

Figure 1 presents the compiler architecture. The frontend performs preprocessing, tokenization, AST construction, and early semantic checking. The middle-end constructs HIR, converts it to SSA [3], runs SSA-level semantic checking with a Z3-backed symbolic layer [4], and applies HIR-level optimizations. The backend lowers the program to LIR, performs LIR data-flow and copy-propagation passes, selects target-sensitive instructions, allocates registers, applies late peephole cleanup, and finally emits assembly for an external assembler/linker toolchain.

This organization deliberately separates source-level analysis, SSA construction, symbolic checking, high-level optimization, lowering, instruction selection, register allocation, and late cleanup. The separation makes the implementation suitable not only for code generation, but also for experiments with IR design, pass placement, and target-specific transformations.

4.2 Worked Example

Listing 14 presents a minimal program used to illustrate the compiler forms.

```
start() {
    i32 a = 1;
    i32 b = 2;
    exit a + b;
}
```

Listing 14: Input CPL program.

The corresponding HIR uses stack variables with an `s` suffix and temporaries with a `t` suffix.

```
fn _main0() {
    i32s %0 = alloc;
    i32t %2 = i8n 1 as i32;
    i32s %0 = i32t %2;
    i32s %1 = alloc;
    i32t %3 = i8n 2 as i32;
    i32s %1 = i32t %3;
    i32t %5 = i32s %0 + i32s %1;
    u8t %4 = i32t %5 as u8;
    exit u8t %4;
}
```

Listing 15: High IR (HIR) form for Listing 14.

After lowering to LIR, the program is represented as a basic-block sequence.

```
BB1: start
%2 = $1 as i32;
%0 = %2;
%3 = $2 as i32;
%1 = %3;
%5 = %0 + %1;
%4 = %5 as u8;
exit %4;
BB2: send
```

Listing 16: LIR form before target-sensitive selection.

A later selected LIR form contains target-dependent register choices.

```
BB1: start
rcx = $1;
rcx = rcx;
rdx = $2;
rdx = rdx;
rax = rcx;
rax = rax + rdx;
rcx = rax;
rcx = rcx;
rdi = rcx;
exit rdi;
BB2: send
```

Listing 17: Selected LIR form.

The final optimized Mach-O assembly for this example collapses the computation to a short sequence.

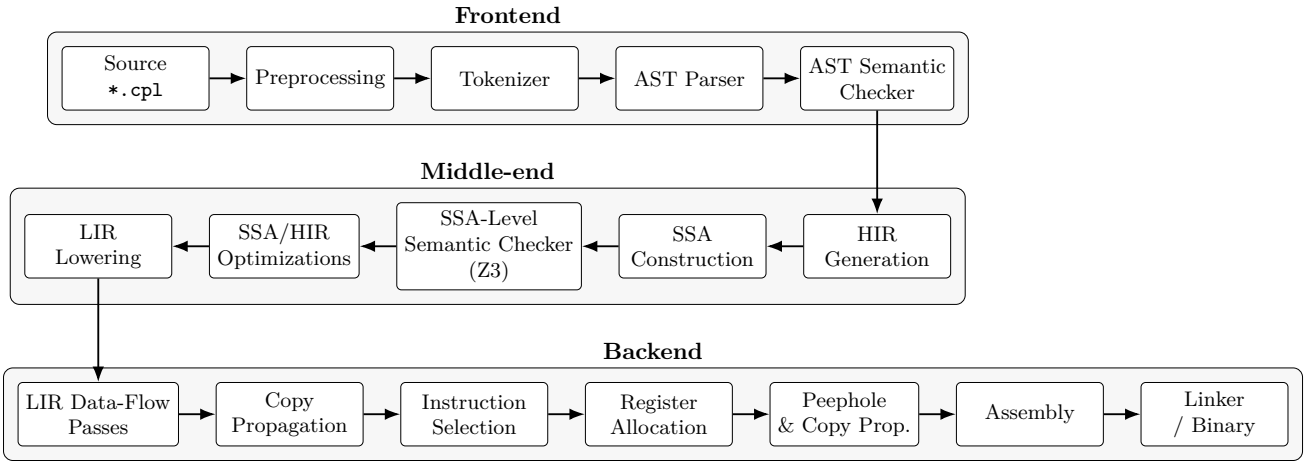

Figure 1: Graphical overview of the CPL compiler architecture. The pipeline is arranged in three phases to preserve readability in a two-column layout while retaining the execution order.

Table 2: Backend maturity in the current CPL compiler snapshot.

Target	Assembler format	Status
x86-64 Mach-O	`macho64`	default; broadest coverage
x86-64 Linux	`elf64`	implemented; targeted tests
i386 Linux	`elf32`	implemented; kernel examples

```
section .text
global _main
_main:
mov al, 1
add al, 2
mov dil, al
mov rax, 0x2000001 ; Exit syscall on MACHO64
syscall
```

Listing 18: Generated assembly.

4.3 Backends and Target Status

The implemented backends in this snapshot are x86_64 Mach-O/NASM, x86_64 GNU/Linux NASM, and i386 GNU/Linux NASM. Each path includes instruction selection, memory selection, caller-saving logic, and assembly generation. Mach-O is the default configuration and has the broadest test coverage. Linux x86_64 and i386 are exercised by dedicated compiler and assembly-generation tests; the i386 path additionally supports kernel-oriented examples. Other architecture and system options exposed by configuration code are outside the evaluated target set. The compiler can invoke `ld`, `clang`, or `gcc` for linking.

4.4 i386 ABI Notes

The i386 backend follows a simple C-style stack ABI for exported functions and extern declarations. Primitive arguments are passed through the caller's stack frame, return values use the target return-register convention, and `extern` declarations describe symbols implemented outside the CPL module. Global CPL functions are emitted as externally visible symbols and can be called from C support code when matching prototypes are used.

The `naked` annotation is the main exception to the ordinary function-frame shape. In a normal i386 function, the compiler may establish an `ebp`-based frame: `[ebp + 4]` contains the return address, `[ebp + 8]` the first argument, and `[ebp + 12]` the second. A naked function suppresses the generated prologue and epilogue, so the corresponding locations are `[esp]`, `[esp + 4]`, and `[esp + 8]`. Figure 2 presents this difference.

```
@[nosection]
@[naked]
glob function i386_switch2user(u32 eip, u32 esp) ->
     i0 {
    asm(eip, esp) {
        "cli",
        "push 0x23",
        "push %1",
        "push 0x202",
        "push 0x1b",
        "push %0",
        "iretd"
    }
}
```

Listing 19: A naked i386 helper whose arguments are addressed from `esp`.

For Listing 19, the backend must materialize the CPL parameters from `[esp + 4]` and `[esp + 8]` before expanding inline-assembly placeholders. Once the assembly body pushes interrupt-frame words, `esp` changes normally; subsequent hand-written instructions must account for that movement.

5 Static Diagnostics

CPL contains a two-level diagnostic subsystem. The first level operates on the AST and targets source-structure issues such as read-only variable updates,

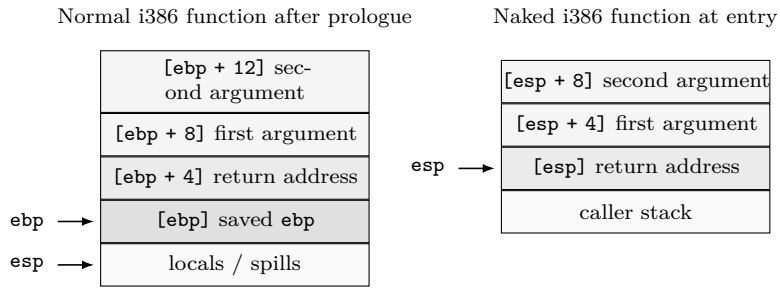

Figure 2: i386 stack view for ordinary and `naked` CPL functions. In this target, ordinary functions use 32-bit registers such as `eax`, `ebx`, `ecx`, `edx`, `esp`, and `ebp`; without `push ebp; mov ebp, esp`, the backend addresses arguments from `esp` rather than from `ebp`.

declaration problems, return mismatches, wrong argument counts, incompatible argument types, unused return values, illegal array access, duplicated branches, invalid function names, dead code, lossy implicit conversions, inefficient infinite loops, invalid `exit` usage, break statements without legal targets, invalid use of `i0`, unused expressions, invalid references, and suspicious alignment requests.

The second level is intentionally placed after SSA construction [3]. At this point the compiler has explicit use-definition chains and phi nodes, so the checker can reason about values and path conditions in a more structured form than in the original HIR. Z3-backed symbolic execution [4] is applied on SSA-form HIR to detect definite null dereferences, null values passed into dereferencing functions, possible null dereferences under path-dependent conditions, and constant branches. This ordering is important: the solver-facing analysis works on SSA-form HIR rather than on pre-SSA HIR, because SSA simplifies symbolic value tracking and makes the generated constraints easier to relate to compiler variables.

The analysis is diagnostic rather than protective. It does not implement memory safety, ownership, or aliasing restrictions, and it does not prevent undefined behavior caused by dangling pointers, unsafe inline assembly, invalid external interfaces, or target-specific misuse.

```
start() {
    function foo(ptr i32 p) -> i32 {
        return dref p; :/ Dereference of NULL /:
    }
    ptr i32 a = 0;
    foo(a);
}
```

Listing 20: Example that triggers SSA-level null-dereference diagnostics.

A representative diagnostic sequence for this example reports both that the return value of `foo(a)` is ignored and that `p` may be dereferenced while equal to null. More complex inputs additionally trigger path-sensitive reports, for example when one branch makes a pointer definitely null while another makes it only conditionally null.

5.1 Z3-Backed Symbolic Layer

The SSA-level checker is implemented as a symbolic layer over the HIR control-flow graph. The analyzer prepares HIR expressions as Z3 formulas, preserves symbolic names for compiler variables, and augments path conditions with phi-node constraints when control-flow edges enter SSA merge points. This design makes it possible to ask local reachability and value questions over the same representation that the optimizer sees.

The current wrapper accepts either parsed JSON or textual HIR dumps, can select a particular function, builds a CFG, and then dispatches solver-backed queries. Two query modes are especially useful for development: `label`, which checks whether a label can be reached under satisfiable path conditions, and `var-eq`, which checks whether a selected variable can, must, or cannot equal a given value. The wrapper also supports initial assignments, pointer-width configuration, strict parsing modes, and cached parsing/analysis artifacts. These features keep the solver path useful not only for compiler diagnostics, but also for investigating IR examples during pass development.

```
analyze-hir --function entry \
    --pointer-width 64 \
    variable-equals temporary_7 0
```

Listing 21: Representative interface to the symbolic query layer.

This subsystem should be understood as an experimental symbolic analysis component, not as a formal verification framework. Z3 is used to answer bounded path and value questions that are helpful for diagnostics such as null-dereference detection and unreachable-code reasoning. It does not by itself provide a complete semantic proof of CPL programs or of compiler transformations [4].

5.2 Scalability

The present evaluation does not isolate solver time or characterize its growth with function size and path count. It also does not use a labeled defect corpus from which false-positive and false-negative rates could be computed. Consequently, RQ3 is answered only at the architectural level: SSA-form HIR can drive path-conditioned Z3 queries, but the speed and diagnostic

accuracy of this design remain unestablished. A quantitative answer requires functions stratified by HIR size and path count, seeded null defects with known ground truth, and separate measurements of solver and total compilation time.

6 Optimization Passes

Table 3 summarizes the passes included in the evaluated optimization profiles. The driver uses `-O0` as the default, enables LICM, constant optimization, and peephole cleanup at `-O2`, and additionally enables LIR copy propagation and tail-recursion elimination at `-O3`. Experimental function inlining is excluded from the evaluated pass set because known correctness defects prevent an interpretable performance claim.

6.1 Loop-Invariant Code Motion

LICM operates on SSA High IR. A loop body that repeatedly computes a constant expression such as `10 + 10` can have this computation moved before the loop, with phi nodes preserving loop-carried values.

```
start() {
    i32 d = 0;
    loop {
        i32 c = 10 + 10;
        d += c;
    }
}
```

Listing 22: LICM source pattern.

6.2 Peephole Optimization

Peephole optimization runs late, after instruction selection and register allocation. It removes redundant register-to-itself moves, simplifies decrement-and-branch sequences, and can replace zero materialization with xor-style idioms where applicable.

```
Before:
rax = rcx;
rax = rax - 1;
rcx = rax;
rdx = rcx;
cmp rdx, 0;
je lb11;
jne lb9;

After:
rcx = rcx - 1;
jne lb10;
```

Listing 23: Peephole effect on a counted empty loop.

6.3 PTRN: A Peephole Pattern DSL

The late peephole pass is not only a collection of hand-written rewrites. CPL also uses `PTRN`, a small domain-specific language for generating peephole optimization patterns. The motivation is practical: low-level instruction cleanup often consists of many small local rewrites, and encoding them in C by hand makes the pass difficult to extend. PTRN provides a declarative syntax for matching short Low LIR instruction sequences and replacing them with simpler or more efficient sequences.

A PTRN file consists of rewrite rules separated by `/`. The left-hand side describes a sequence to match; the right-hand side after `->` describes the replacement. Pattern objects abstract over concrete Low LIR operands: `areg_N` tracks a repeated abstract register, `aconst_N` tracks a repeated abstract constant, `mem_N` matches memory locations, and `obj` can match an arbitrary object. Conditions such as `[if:equals]`, `[if:arg2:zero]`, or `[if:arg2:mod2]` restrict a match, while actions can transform matched constants in the replacement.

```
; Remove a jump that immediately targets the next
    label.
jmp label_1
mklb label_1
->
mklb label_1
/

; Remove redundant self-copy.
mov obj_1, obj_2 [if:equals]
->
delete
/

; Prefer xor-zeroing over moving an immediate zero.
mov areg_1, const 0
->
xor areg_1, areg_1
/
```

Listing 24: Examples of PTRN peephole rules.

The generated C code is then used by the low-level peephole pass. This keeps the compiler backend extensible: adding a new local simplification can be done by adding a pattern rule rather than modifying the peephole engine directly. The approach is intentionally modest; PTRN does not replace global optimization or data-flow analysis. Its role is to make late instruction cleanup explicit, testable, and easier to maintain.

The report does not attribute an independent performance improvement to PTRN because the available benchmark data measures the complete optimization pipeline. PTRN is therefore evaluated here as an implementation mechanism for declarative and generated local rewrites, rather than as an isolated source of speedup.

6.4 Excluded Experimental Passes

The implementation contains heuristic and model-guided function-inlining experiments. They are not part of the evaluated `-O0`/`-O2`/`-O3` evidence in this report. Known call-site rewriting defects and the absence of model, dataset, and accuracy documentation make performance results involving these modes uninterpretable. They are therefore treated as implementation prototypes rather than research results.

Table 3: Optimization passes implemented in the current CPL compiler.

Pass	IR level and role	Level
LICM	Works on SSA HIR; hoists loop-invariant expressions after loop canonicalization.	`-O2` and above
Peephole	Works on selected LIR after instruction selection, register allocation, memory selection, and caller-saving insertion.	`-O2` and above
DAG rebuild and sparse constant propagation	Works on HIR via DAG generation and CFG rebuild before lowering to LIR.	`-O2` and above
Dead-function elimination	Builds and applies a call graph before later HIR transformations.	default pipeline
LIR copy propagation and unused-variable dropping	Rewrites propagated LIR operands and removes unused variable writes.	`-O3`
Tail-recursion elimination	Works on HIR and rewrites tail self-calls to loops before SSA construction.	`-O3`
Hot/cold placement	Implemented during AST-to-HIR generation as annotation-driven branch layout.	default pipeline

7 Examples and Use Cases

The evaluated examples exercise core low-level features rather than large application workloads. Listings are shortened to isolate the language mechanisms relevant to each case.

Table 4: Representative CPL programs.

Program	What it demonstrates
Hello World	Strings, pointers to string literals, `strlen`, inline assembly, direct syscall use, and entry-point termination.
CRC8	Global arrays, indexed memory access, loops, pointer arguments, integer operations, and exit-code calculation.
Brainfuck interpreter	Arrays, argument access, switch dispatch, `no_fall`, `straight`, loops, function calls, and byte-level tape manipulation.
Memory and file helpers	Header-style declarations, syscall wrappers, pointer arithmetic, and low-level I/O abstractions.
OS kernel helpers	i386 extern boundaries, interrupt-related routines, keyboard-driver logic, and C ABI integration.
Multiboot kernel entry	Packed header layout, aligned sections, linker-visible entry points, register binding, stack construction, and transfer to a higher-level kernel routine.

Listing 25 presents a compact Hello World excerpt using a direct syscall wrapper. The complete version includes the full `strlen` implementation and register preservation sequence.

```
function puts(ptr i8 s) -> i0 {
    asm (s, strlen(s)) {
        "mov rax, 33554436",
        "mov rdi, 1",
        "mov rsi, %0",
        "mov rdx, %1",
        "syscall"
    }
}
```

Listing 25: Shortened Hello World with explicit syscall wrapper.

7.1 Operating-System Code on i386

A practical use case for CPL is small freestanding routines used by an operating-system kernel. The i386 backend was added to support this style of code: globally visible routines can be emitted as NASM-compatible 32-bit assembly, while `extern` declarations make it possible to integrate CPL modules with C kernel code.

Listing 26 presents a shortened PS/2 keyboard driver excerpt. The complete example additionally contains scancode tables, polling helpers, initialization code, and an exported C boundary.

```
extern function i386_inb(u16 port) -> i8;
extern function i386_outb(u16 port, u8 data) -> i0;
extern function i386_irq_registerHandler(i32 irq, ptr
      i0 handler) -> i0;

glob arr _key_pressed[128, u8] = { 0 };

function i386_keyboard_handler(ptr i0 _) -> i0 {
    i8 character = i386_inb(0x60);
    if character < 0 || character >= 128; return;
    _key_pressed[character as i32] = 1;
}
```

Listing 26: Shortened i386 PS/2 keyboard driver excerpt using C externs.

Imported and exported symbols form ordinary low-level ABI boundaries. In this organization, CPL expresses driver control flow and data structures, while C or platform support code supplies target-specific primitives.

7.2 Multiboot Kernel Bootstrap

A second i386 use case is replacement of a conventional assembly bootstrap with a mostly typed CPL translation unit. Multiboot version 0.6.96 requires a 32-bit-aligned header within the first 8192 bytes of the operating-system image. The header begins with the magic value `0x1BADB002`; its magic, flags, and checksum fields must sum to zero modulo 2^{32}. At transfer of control, `EAX` contains the Multiboot boot-loader magic, `EBX` points to the boot-information structure, and the operating system must establish its own stack [5].

Listing 27 expresses these requirements through packed containers, explicit section placement and alignment, a linker-visible entry symbol, and register-bound source variables. Inline assembly remains necessary for instructions whose effects are not represented by ordinary CPL expressions, including loading `esp`, disabling interrupts, and halting the processor. The header layout, static storage, and call into the kernel routine remain visible to the type system and normal compiler pipeline.

```
@[align(1)]
container multiboot_header {
    u32 magic;
    u32 flags;
    u32 checksum;
    arr reserved[5, u32];
    u32 mode;
    u32 width;
    u32 height;
    u32 depth;
}

@[section(".multiboot", 4)]
glob multiboot_header header = {
    :/ magic    /: 0x1BADB002,
    :/ flags    /: 7,
    :/ checksum /: 3830599671,
    :/ reserved /: 0, 0, 0, 0, 0,
    :/ mode     /: 0,
    :/ width    /: 640,
    :/ height   /: 480,
    :/ depth    /: 32
};

container kernel_stack {
    arr storage[16384, u8];
}

@[section(".bss", 16)]
glob kernel_stack stack;

@[section(".text")]
function kmain(u32 info, u32 magic, u32 stack_top) -> i0;

@[section(".text")]
@[entry("_start")]
@[naked]
function main() -> i0 {
    @[register(4)] u32 magic;
    @[register(5)] u32 info;

    asm(magic, info) {
        "mov %0, eax",
        "mov %1, ebx"
    }
    asm(ref stack + sizeof(kernel_stack)) {
        "mov esp, %0",
        "cli",
        "xor ebp, ebp"
    }

    u32 stack_top = 0;
    asm(stack_top) { "mov %0, esp" }
    kmain(info, magic, stack_top);
    asm() { ".hang: hlt", "jmp .hang" }
}
```

Listing 27: Abridged Multiboot-compatible i386 entry unit.

The case study does not eliminate assembly semantically; privileged and register-specific operations remain explicit. It instead narrows handwritten assembly to the target operations that require it, while representing binary layout, symbol placement, initialization, and control transfer in a typed source language. This division is useful for boot code because layout constraints remain auditable without forcing the complete entry path into an untyped assembly file.

8 Testing Infrastructure

The compiler is tested with a phase-oriented framework rather than only through end-to-end examples. The central principle is phase observability: tests can inspect preprocessing, tokenization, AST construction, semantic analysis, HIR generation, SSA construction, DAG construction, constant-folding analysis, LIR construction, instruction selection, register allocation, peephole optimization, assembly generation, or assembly-level constant folding. This makes regressions easier to localize than in a purely black-box compiler test.

CPL tests use an `OUTPUT` oracle embedded in source files. The oracle can match runtime output, exit codes, and selected compiler dumps. It also supports tolerant matching for unstable compiler-generated identifiers, so tests can focus on semantically important output instead of on incidental temporary names. Runtime tests can run generated assembly and can use multiple argument cases inside one file.

Two flags are especially important for the current workflow. `BUG` marks an expected failing test and keeps known defects visible without making the entire test run unusable. `LEAK_TRACE` enables memory-operation logging for leak localization in compiler-internal tests. Together with phase observability, these mechanisms support regression tracking across frontend, middle-end, backend, and runtime behavior.

The regression corpus covers simple output programs, counted loops, CRC-style table traversal, arithmetic and function-call kernels, Fibonacci, switch dispatch, and a larger Brainfuck interpreter. It also includes OS-oriented i386 examples. This testing setup is not a proof of correctness, but it checks internal forms and executable behavior and provides a practical mechanism for detecting pass-level regressions.

9 Evaluation and Evidence

The evaluation combines one qualitative systems case study with a preliminary microbenchmark study. Table 6 states which research questions receive evidence and prevents unmeasured properties from being inferred from implementation descriptions.

The chosen kernels expose loop overhead, arithmetic simplification, and pointer-heavy traversal. They are useful for identifying workload-specific behavior but are not a substitute for a standard benchmark suite or kernel-level performance comparison.

All available timings were collected on macOS Catalina 10.15.7 using an Intel i7-3650QM CPU at 2.4 GHz and 16 GB DDR3 memory. The compared tools were GCC 14.2.0, Apple Clang 12.0.0, a Mach-O Cordell compiler build, and rustc 1.87.0. GCC and Clang were tested at `-O0` and `-O3`, CPL at `-O0` and `-O3`, and Rust at `opt-level=0` and `opt-level=2`. Each reported value is the arithmetic mean of five runs after compilation. Raw repetitions and dispersion statistics

Table 5: Compiler phases exposed by the integration testing framework.

Frontend	HIR / SSA	LIR / backend	Assembly
`preproc`	`hir`	`lir`	`asm`
`prep`	`hir_ssa`	`lir_constfold`	`asm_constfold`
`ast`	`hir_dag`	`lir_selector`	
`sem`	`hir_constfold`	`lir_instplan`	
		`lir_regalloc`	
		`lir_peephole`	

Table 6: Evidence available for the research questions.

RQ	Evidence	Supported conclusion	Status
RQ1	Multiboot entry unit and i386 kernel helpers	Typed CPL represents layout, sections, symbols, stack storage, and higher-level control transfer; machine-register and privileged operations still require assembly.	partially answered
RQ2	Three five-run microbenchmarks against Clang, GCC, and Rust	Arithmetic-loop results are close to Clang/GCC; pointer traversal is slower than every baseline.	preliminary answer
RQ3	Implemented SSA-to-Z3 query layer and diagnostic examples	The representation supports path-conditioned null and reachability queries; precision and cost are unknown.	architectural only
RQ4	`-O0`/`-O3` aggregate comparison	Optimization profiles affect runtime, but individual pass contributions and code-size effects cannot be identified.	unanswered

were not retained; the table therefore does not support statistical significance claims. The measurements also use different timing wrappers for CPL and the C-family baselines and must be interpreted conservatively. The empty-loop benchmark is a loop-overhead test: the C baseline uses `asm volatile` to prevent deletion of the loop.

9.1 Benchmarked Kernels

Listing 28 presents the counted empty loop used to evaluate loop overhead and late low-level simplification.

```
@[naked] start() {
    @[counter(1000000000)] loop {
    }
    exit 0;
}
```

Listing 28: Empty-loop benchmark kernel.

Listing 29 presents the Fibonacci kernel used to exercise simple arithmetic and loop-carried values.

```
start() {
    i32 a = 1;
    i32 b = 0;
    @[counter(1000000)] loop {
        i32 tmp = a;
        a = a + b;
        b = tmp;
    }
    exit b;
}
```

Listing 29: Fibonacci benchmark kernel.

Listing 30 presents the string-traversal kernel, which is more representative of pointer-based code generation.

```
start() {
    str msg = "abcdefghijklmnopqrstuvwxyzABCDEFGHIJKLMNOPQRSTUVWXYZ0123456789+/";
    i64 outer = 0;
    u8 acc = 0;
    ptr i8 p = ref msg;

    while outer < 1000000; {
        p = ref msg;
        while dref p; {
            acc = (acc + dref p) & 0xFF;
            p += 1;
        }
        outer += 1;
    }

    exit acc;
}
```

Listing 30: String-traversal benchmark kernel.

9.2 Optimized Runtimes

Table 7 reports the optimized runtimes. Figure 3 visualizes the same measurements.

For RQ2, CPL is within 1.2% of the fastest Clang/GCC result on Fibonacci and is 6.1–7.4% faster than those two C compilers on the empty-loop measurement. Rust is substantially faster on both kernels. On string traversal, CPL is 9.1% slower than GCC, 19.3% slower than Clang, and 54.5% slower than Rust. The evidence therefore supports a narrow statement: the current backend approaches mature C compilers on the measured scalar loops but underperforms on pointer-heavy traversal. Possible causes include additional pointer moves, weaker address-generation simplification, and missed induction-variable transformations. These explanations are hypotheses until tested using decoded instruction counts and hardware counters.

Table 7: Mean optimized benchmark runtimes in seconds over five runs. Lower is better; dispersion data is unavailable.

Benchmark	CPL `-O3`	Clang `-O3`	GCC `-O3`	Rust opt-level=2
Empty loop	0.702	0.748	0.758	0.333
Fibonacci	0.412	0.417	0.412	0.335
String traversal	0.513	0.430	0.470	0.332

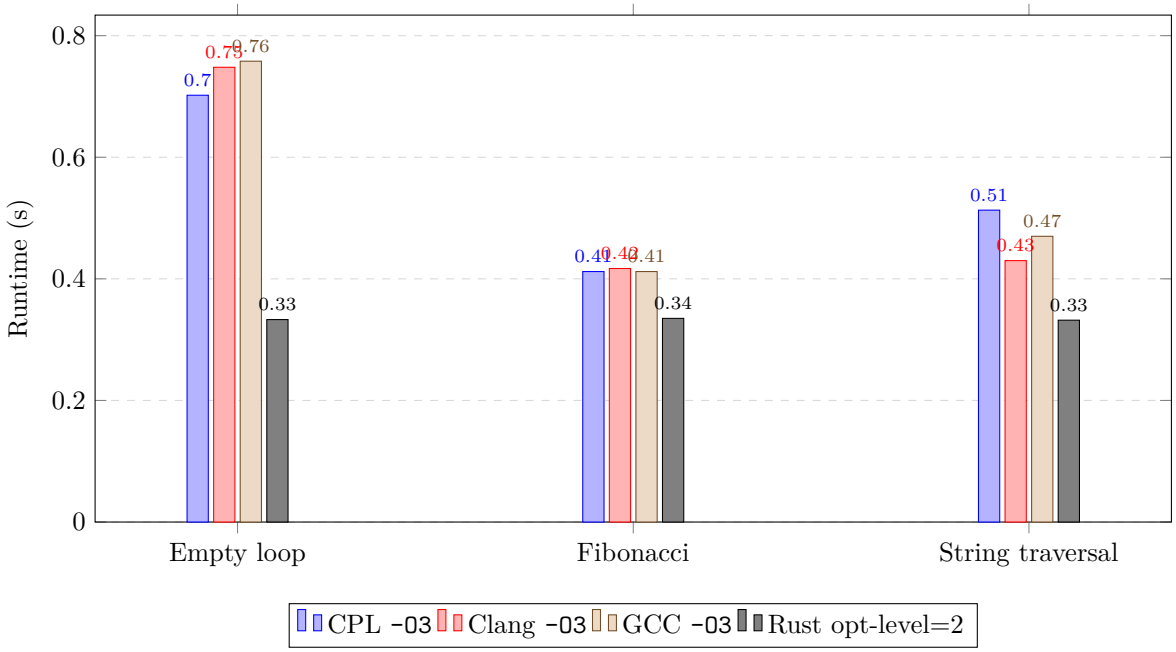

Figure 3: Optimized runtimes for the three benchmark kernels.

9.3 Effect of Optimization within CPL

Figure 4 presents the effect of CPL optimization alone by comparing `-O0` and `-O3`. The empty loop benefits the most, which is consistent with the late low-level simplifications described earlier. Within CPL, LICM affects loop-invariant HIR expressions, peephole cleanup removes redundant selected-LIR instructions, and copy propagation reduces low-level temporary traffic after lowering. Because no pass-by-pass ablation was performed, the observed improvement cannot be assigned to an individual optimization.

Generated-code size was not measured in object bytes or decoded instruction counts. Source-level or assembly-line counts are omitted because directives, labels, and formatting make them an unreliable proxy for machine-code size.

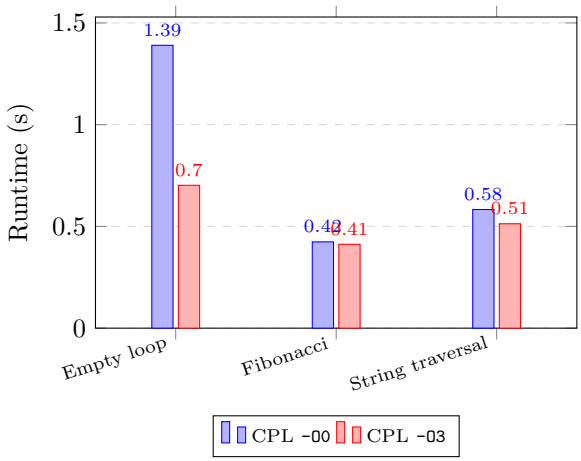

Figure 4: Optimization effect within CPL.

10 Limitations and Threats to Validity

The results are subject to the following limitations.

Partial formalization only. Section 3.9 defines typing and reduction rules for a small pointer-and-container core. It does not cover the complete grammar, target annotations, inline assembly, or every control-flow construct, and it is not mechanized. The compiler has no semantic-preservation proof or translation validation.

No memory safety. The static analyzer can find selected problems, but CPL intentionally does not implement an ownership or borrowing discipline.

Limited target validation. The most exercised path is x86_64 Mach-O/NASM. x86_64 GNU/Linux NASM is implemented but less tested. The i386 GNU/Linux NASM backend has dedicated assembly-

generation tests and real operating-system helper examples, but it is newer and has not yet been evaluated with the same breadth as the Mach-O path. Other architecture and system-type options exposed by the driver should be treated as planned or partial support unless backed by dedicated tests.

Excluded function-inlining prototypes. Heuristic and model-guided inlining modes exist in the implementation but are excluded from the evaluated optimization profiles and contribution claims because their correctness and effectiveness have not been established.

Small benchmark suite. The benchmark suite consists of microbenchmarks and small interpreters. It does not include large applications, compiler self-hosting, SPEC-style workloads, kernel compilation workloads, broad target comparisons, or Linux x86_64 replication.

Single-machine measurements. Measurements were performed on a single machine, macOS 10.15.7 with an Intel i7-3650QM. Results may vary across processors, operating systems, assemblers, linkers, and timing wrappers.

No cache or memory-hierarchy analysis. The current evaluation does not include cache-miss counters, branch-misprediction counters, memory-bandwidth measurements, or other memory-hierarchy analysis. Pointer-heavy results such as string traversal should therefore be interpreted cautiously.

No differential fuzzing campaign. The phase-oriented regression suite checks known examples and internal representations, but it does not provide randomized differential validation against a reference interpreter or a semantically equivalent C subset. Compiler reliability beyond the covered tests is therefore not quantified.

Evaluation snapshot. The benchmark data was collected for a particular Mach-O compiler snapshot and hardware configuration. It should therefore be treated as preliminary evidence for the implementation direction rather than as a complete evaluation of every backend and optimization combination.

Inline assembly opacity. Inline assembly is copied into the output after placeholder substitution and is not optimized by the compiler. This can invalidate assumptions made by surrounding optimization passes if the programmer uses labels, jumps, or unpreserved registers in fragile ways.

11 Related Work

Table 8 compares CPL with projects that represent distinct points in the design space: LLVM [9], QBE [1], TinyCC [2], Zig [18], CompCert [10, 16], Cogent [11], and Low* [12]. The comparison separates language scope, primary objective, low-level access, formal assurance, and backend maturity rather than treating all systems as interchangeable compiler projects.

The closest implementation-scale comparators are QBE and TinyCC, but each leaves a different gap. QBE provides a compact backend rather than a source language with typed OS-facing constructs. TinyCC provides practical C compatibility, but inherits the full complexity and expectations of C and is not organized as a small SSA/SMT experimentation platform. Zig provides substantially stronger language and ecosystem support, while CompCert, Cogent, and Low* provide assurance that CPL does not claim.

The resulting niche is narrow: CPL combines an inspectable source language, explicit boot- and kernel-facing controls, a multi-stage optimizing pipeline, and an experimental SSA/SMT diagnostic layer in one compact implementation. This is a research gap only in the sense of an engineering combination, not evidence of superiority. CPL currently lacks the formal assurance of verified systems, the validation methodology exemplified by Csmith [17], and the benchmark breadth and backend maturity of production compilers. The report's contribution is therefore the design and feasibility evidence for that combination.

12 Future Work

A dissertation-strength evaluation requires the following work:

- extend RQ2 to at least 10–15 reproducible kernels, including `strlen`, `memcpy`, CRC variants, sorting, matrix, and kernel-support workloads;
- answer RQ4 with `-O0`/`-O2`/`-O3` measurements and one-pass-at-a-time ablations for LICM, constant propagation, copy propagation, and peephole/PTRN rewriting;
- record object bytes, decoded instruction counts, cache misses, and branch mispredictions in addition to wall-clock time;
- evaluate RQ3 on a labeled corpus of seeded and real pointer defects, reporting solver time, total compile time, false positives, and false negatives;
- validate the i386 path by booting the generated Multiboot artifact and exercising several kernel subsystems under a reproducible emulator configuration;
- mechanize the core semantics and extend it to control flow, calls, arrays, annotations, and target interactions;
- conduct randomized and differential testing against a reference interpreter or a rigorously defined translatable subset;

Table 8: Positioning of CPL relative to representative systems languages and compiler infrastructures.

Project	Scope	Primary objective	Low-level facilities	Formal assurance	Backend maturity
LLVM	reusable compiler infrastructure	broad optimization and target support	low-level IR; frontend-dependent source facilities	no general frontend correctness proof	production, many targets
QBE	compact SSA backend	simple reusable code generation	low-level IR, not a systems source language	none claimed	small but established backend
TinyCC	compact C compiler	fast compilation and C compatibility	C pointers, ABI and system interfaces	none claimed	practical multi-platform C compiler
Zig	full systems language	production systems development	explicit memory, ABI, inline assembly, compile-time facilities	language safety checks, no verified compiler claim	production-oriented, broad targets
CompCert	C compiler	semantic preservation	C-level systems facilities	machine-checked compiler proof	mature verified target set
Cogent/Low*	restricted verified systems languages	proof-oriented low-level software	controlled memory and C interoperability	mechanized semantics and proofs	specialized verified toolchains
CPL	compact source language and compiler	inspectable OS/compiler experiments	pointers, layout, sections, entry points, syscalls, inline assembly	partial paper model; no proof	three x86-family formats, uneven validation

- keep function inlining outside performance claims until correctness regressions are resolved; evaluate learned inlining only after publishing its features, dataset, model, and decision metrics.

Classes, enums, constructors, destructors, inheritance, and ownership-oriented aggregate models remain outside the current core by design. Containers provide a deliberately small C-like aggregate mechanism, but richer user-defined type systems may be studied separately. This report treats that restriction as part of CPL's deliberate language model rather than as an accidental omission.

13 Conclusion

RQ1 is supported by the Multiboot case study: CPL represents binary layout, aligned sections, symbols, stack storage, and transfer to typed kernel code, while register capture and privileged instructions remain explicit assembly. RQ2 receives only preliminary support: CPL approaches Clang and GCC on the measured scalar loops but is slower on pointer traversal and substantially behind Rust in all three measurements. RQ3 is answered architecturally rather than quantitatively: SSA-form HIR supports path-conditioned Z3 queries, but diagnostic precision and cost are unknown. RQ4 remains unanswered because no pass-level ablation or machine-code-size study was performed.

The report therefore establishes feasibility of a compact source language that combines OS-facing controls with a conventional optimizing pipeline and SSA/SMT analysis. It does not establish compiler correctness, memory safety, general performance competitiveness, backend maturity, or diagnostic effectiveness. Those claims require the formalization, differential validation, expanded benchmarks, hardware-counter measurements, and labeled diagnostic evaluation specified in the future-work program.

A Abridged BNF Grammar of CPL

This appendix gives an abridged grammar reconstructed from the parser implementation in the evaluated snapshot. It documents principal forms rather than a complete formal operational semantics.

```
program        ::= top_item*
top_item       ::= annotation* (start_decl |
    function_decl | container_decl
                 | extern_decl | import_decl |
    section_decl | align_decl
                 | var_decl | pp_directive | block)

start_decl     ::= "start" "(" param_list? ")" block
function_decl  ::= "function" ident generic_params?
    "(" param_list? ")"
                   ("->" type)? (";" | block)
extern_decl    ::= "extern" (function_decl | type
    ident ";")
import_decl    ::= "from" string "import" ident ("," 
    ident)* ";"?

container_decl ::= "container" ident "{"
    container_item* "}"
container_item ::= annotation* (field_decl |
    method_decl | pp_directive)
field_decl     ::= storage_mod* annotation* type
    ident ("=" expr)? stmt_end
method_decl    ::= "function" ident generic_params?
    "(" param_list? ")"
                   ("->" type)? (";" | block)

param_list     ::= param ("," param)*
param          ::= annotation* ("..." type? ident? |
    "self" | type annotation* ident ("=" expr)?)
generic_params ::= "<" ident ("," ident)* ">"

storage_mod    ::= "glob" | "ro"
type           ::= prim_type | "ptr" type | "arr" "["
     const_len "," type "]"
                 | "(" type_list? ")" "=>" type |
    ident
prim_type      ::= "f64" | "f32" | "i64" | "i32" | "
    i16" | "i8"
                 | "u64" | "u32" | "u16" | "u8" | "i0
    " | "str"
const_len      ::= int | numeric_macro
```

```
statement      ::= block | function_decl | align_decl
      | start_decl
                 | if_stmt | while_stmt | loop_stmt |
      switch_stmt
                 | return_stmt | exit_stmt |
     break_stmt | lis_stmt
                 | syscall_stmt | asm_stmt | var_decl
      | expr ";"
block          ::= "{" statement* "}"
if_stmt        ::= "if" expr ";" statement ("else"
     statement)?
while_stmt     ::= "while" expr? ";" statement
loop_stmt      ::= "loop" statement
switch_stmt    ::= "switch" expr ";" "{" case_clause*
      default_clause? "}"
case_clause    ::= "case" literal ";" block
default_clause ::= "default" ";"? block

var_decl       ::= storage_mod* annotation* (
     array_decl | type ident ("=" expr)? stmt_end)
array_decl     ::= "arr" ident "[" const_len "," type
      "]" ("=" array_init)? stmt_end
                 | type ident "[" const_len "]" ("="
     array_init)? stmt_end

annotation     ::= "@" "[" annotation_text "]"
pp_directive   ::= "#" ("line" | "include" | "define"
      | "undef"
                 | "ifdef" | "ifndef" | "endif") ...
     line_end
stmt_end       ::= ";" | end_of_line
```

Listing 31: Abridged CPL grammar.

The current keyword set is: `start`, `exit`, `function`, `container`, `return`, `if`, `else`, `while`, `loop`, `switch`, `case`, `default`, `glob`, `ro`, `dref`, `ref`, `ptr`, `lis`, `break`, `extern`, `from`, `import`, `syscall`, `asm`, `as`, `f64`, `f32`, `i64`, `i32`, `i16`, `i8`, `u64`, `u32`, `u16`, `u8`, `i0`, `str`, `arr`, `not`, `neg`, `poparg`, `sizeof`, `section`, `align`, `line`, `include`, `define`, `undef`, `ifdef`, `ifndef`, and `endif`. Containers may contain scalar fields, pointer fields, and array fields; method-like calls are supported through explicit receiver parameters marked with `@[self]`. The `@[like_c]` annotation is used to document C-ABI-like container padding and layout intent.

References

[1] Quentin Carbonneaux Babiak and contributors. *QBE: A Quick Backend*, 2026. Accessed 2026-05-30.

[2] Fabrice Bellard and TinyCC contributors. *TinyCC: Tiny C Compiler*, 2026. Accessed 2026-05-30.

[3] Ron Cytron, Jeanne Ferrante, Barry K. Rosen, Mark N. Wegman, and F. Kenneth Zadeck. Efficiently computing static single assignment form and the control dependence graph. *ACM Transactions on Programming Languages and Systems*, 13(4):451–490, 1991.

[4] Leonardo de Moura and Nikolaj Bjørner. Z3: An efficient SMT solver. In *Tools and Algorithms for the Construction and Analysis of Systems*, pages 337–340. Springer, 2008.

[5] Bryan Ford, Erich Stefan Boleyn, and Free Software Foundation. *Multiboot Specification, Version 0.6.96*. GNU Project, 2010. Accessed 2026-06-13.

[6] Ralf Jung, Jacques-Henri Jourdan, Robbert Krebbers, and Derek Dreyer. RustBelt: Securing the foundations of the Rust programming language. *Proceedings of the ACM on Programming Languages*, 2(POPL):66:1–66:34, 2018.

[7] Brian W. Kernighan and Dennis M. Ritchie. *The C Programming Language*. Prentice Hall, 2 edition, 1988.

[8] Steve Klabnik, Carol Nichols, and Chris Krycho. *The Rust Programming Language*. No Starch Press, 2 edition, 2023.

[9] Chris Lattner and Vikram Adve. LLVM: A compilation framework for lifelong program analysis and transformation. In *Proceedings of the International Symposium on Code Generation and Optimization*, pages 75–88, 2004.

[10] Xavier Leroy. Formal verification of a realistic compiler. *Communications of the ACM*, 52(7):107–115, 2009.

[11] Liam O'Connor, Christine Rizkallah, Zilin Chen, Sidney Amani, Japheth Lim, Yutaka Nagashima, Thomas Sewell, Alex Hixon, Gabriele Keller, Toby Murray, and Gerwin Klein. COGENT: Certified compilation for a functional systems language, 2016.

[12] Jonathan Protzenko, Jean-Karim Zinzindohoué, Aseem Rastogi, Tahina Ramananandro, Peng Wang, Santiago Zanella-Béguelin, Antoine Delignat-Lavaud, Cătălin Hriţcu, Karthikeyan Bhargavan, Cédric Fournet, and Nikhil Swamy. Verified low-level programming embedded in F*. *Proceedings of the ACM on Programming Languages*, 1(ICFP):17:1–17:29, 2017.

[13] D. M. Ritchie, S. C. Johnson, M. E. Lesk, and B. W. Kernighan. UNIX time-sharing system: The C programming language. *The Bell System Technical Journal*, 57(6):1991–2019, 1978.

[14] Bjarne Stroustrup. Evolving a language in and for the real world: C++ 1991–2006. In *Proceedings of the Third ACM SIGPLAN Conference on History of Programming Languages*, HOPL III, pages 4–1–4–59. ACM, 2007.

[15] Bjarne Stroustrup. *The C++ Programming Language*. Addison-Wesley, 4 edition, 2013.

[16] The CompCert Project. *The CompCert Verified Compiler*, 2026. Accessed 2026-05-12.

[17] Xuejun Yang, Yang Chen, Eric Eide, and John Regehr. Finding and understanding bugs in C compilers. In *Proceedings of the 32nd ACM SIGPLAN Conference on Programming Language Design and Implementation*, pages 283–294. ACM, 2011.

[18] Zig Software Foundation. *The Zig Programming Language Reference*, 2026. Accessed 2026-05-12.